

Predicción de Buy Through Rate (BTR) con Transformers

TP1 — Transformers

Katia Menshikoff, Agostina Squillari y Javier Peral Belmont

Objetivo

- Predecir el Buy Through Rate (BTR) en un e-commerce de supermercado: productos comprados / productos impresos en resultados de búsqueda.
- Dataset: supermarket_products.csv — eventos de búsqueda (compras, vistas, interacciones con filtros).
- Ejercicio 1 — Formulación del problema y EDA
- Ejercicio 2 — Desarrollo del sistema
- Ejercicio 3 — Personalización (teórico)

Ejercicio 1

Formulación del problema y EDA

Punto de partida del dataset

- 10.000 filas, 22 columnas.
- Cada fila NO es una búsqueda completa — es un producto puntual mostrado como resultado dentro de una búsqueda (query_id).
- 2.012 búsquedas (query_id) en total, entre 1 y 8 productos impresos por búsqueda (~5 en promedio).
- Columnas cart y bought indican, por fila, si ESE producto en ESA búsqueda se agregó al carrito / se compró.

Variable objetivo: bought a nivel fila

- El BTR es una métrica agregada a nivel búsqueda, pero el dataset viene desagregado a nivel fila-producto-búsqueda.
- Variable objetivo elegida: bought, a nivel fila — probabilidad de que ESE producto puntual sea comprado.
- El BTR de una búsqueda sale de promediar bought (o las probabilidades predichas) agrupando por query_id.

Relación cart y bought

cart	bought	% filas
True	True	13,01%
True	False	17,06%
False	True	0,00%
False	False	69,93%

- bought=True implica siempre cart=True (0% de compras sin carrito).
- No al revés: de los que llegaron al carrito (30,07%), solo el 43,3% terminó comprado.
- bought = True: 13,01% de las filas.
- cart = True: 30,07% de las filas.

Conclusión: cart no puede usarse como feature — al momento de predecir (búsqueda mostrada) todavía no se sabe si se va a agregar al carrito.

Nulos en allergens (única columna con nulos, 44,55%)

- Nulo siempre (100%): Household, Meat, Personal Care, Produce — no aplica declarar alérgenos alimentarios.
- Nunca nulo (0%): Bakery, Dairy, Seafood.
- Parcial: Beverages (50,8%), Baby (34,0%), Frozen (25,2%), Pantry (25,1%), Snacks (18,6%).
- No es aleatorio: depende 100% de category.

Conclusión: el nulo codifica información real ("sin alérgenos declarados"), no un dato perdido — tratarlo como categoría propia al encodear, no imputar ni descartar.

- Se probó si el alérgeno declarado aparece como substring en la lista de ingredientes de la misma fila.
- La misma combinación exacta de ingredientes aparece unas veces con un alérgeno y otras con otro (ej. "Prepared ingredients, Spices, Salt" con Wheat y con Soy).
- % de coincidencia: 0% para Fish, Peanuts, Shellfish, Soy, Tree nuts; parcial (55-68%) solo para Wheat/Milk (por casualidad de palabras como "Wheat flour").

Conclusión: allergens se generó de forma independiente de ingredients (probablemente condicionado solo por category, igual que los nulos) — no se puede usar una para derivar la otra.

aclarar que aca usamos one-hot encoding

dimensions_in: sin estándar de envase

- Formato: "3.3 x 4.0 x 4.1\" (largo x ancho x alto, pulgadas).
- Hipótesis: no hay estándar de envase, cada producto tiene su propia medida.
- Volumen vs net_weight_oz: correlación 0,38 (moderada-baja, no exacta).
- Solo 136 de 10.000 filas con dimensions_in exactamente repetido.

Conclusión: no tratar como categórica (quedaría con ~9864 categorías) — parsear a 3 numéricas (largo, ancho, alto) o resumir en volumen.

package_size / unit_of_measure vs net_weight_oz

- unit_of_measure: oz, ct, lb, gal, fl oz. package_size: texto "10 oz", "12 ct", etc.
- Aún con unit_of_measure='oz', el número de package_size casi nunca coincide exacto con net_weight_oz (0,4% de match exacto).
- Para ct/lb/gal/fl oz no hay fórmula de conversión limpia (mismo package_size, distinto net_weight_oz).

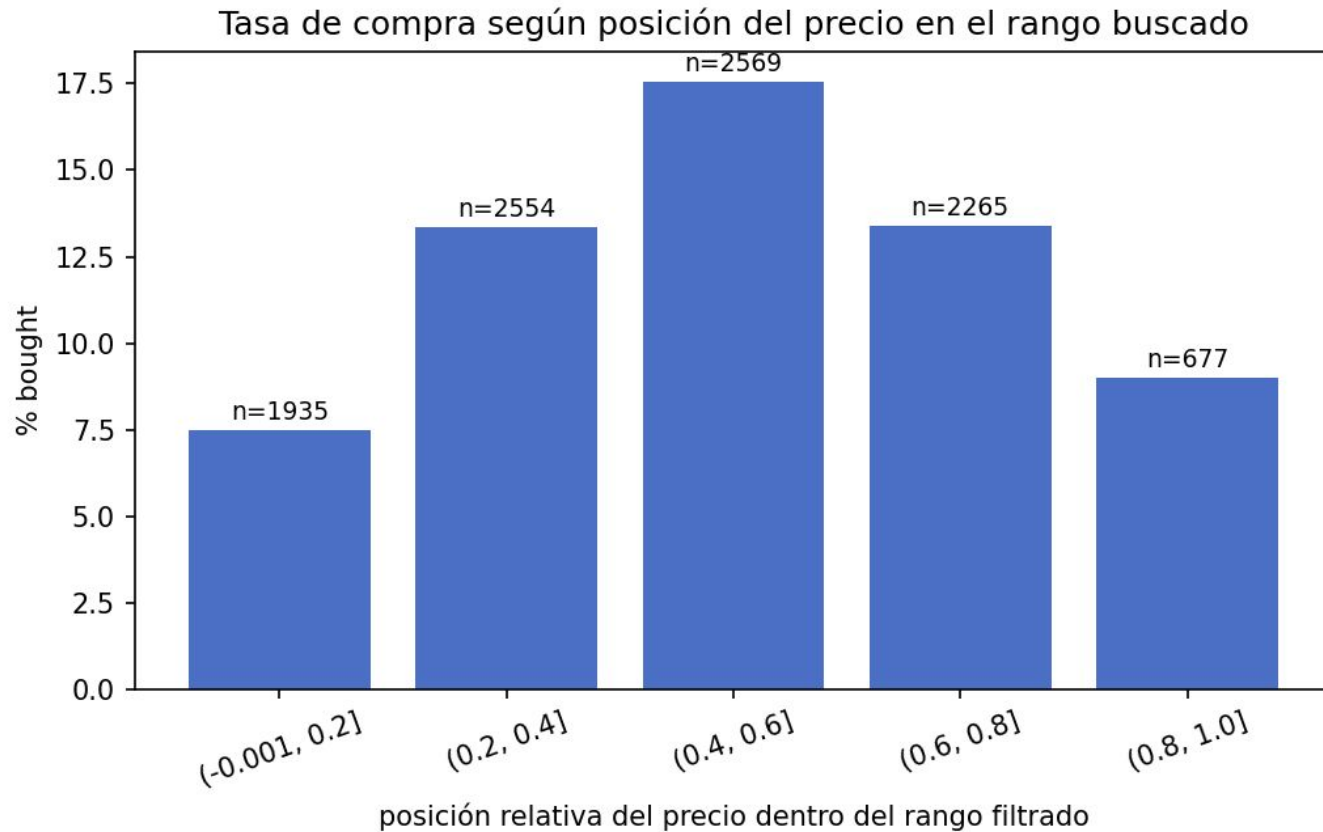
Conclusión: package_size y unit_of_measure son redundantes con net_weight_oz (ya normalizado a onzas) — se descartan como features.

ingredients: 190 valores → 12 combinaciones reales

- n unique crudo: 190. Ordenando los ingredientes dentro de cada fila (ignorando orden): solo 12 combinaciones reales.
- Vocabulario total: 21 ingredientes individuales distintos.

Conclusión: no tratar como texto libre ni 190 categorías. Dos caminos: multi-hot sobre los 21 ingredientes, o categórica de 12 combinaciones canónicas.

price vs. filter_price_min / filter_price_max



Parámetros de la corrida:

- $pos = (price - filter_price_min) / (filter_price_max - filter_price_min)$
- bins de 0.2 en 0.2, sobre 10.000 filas

- price siempre cae dentro de $[filter_price_min, filter_price_max]$ — 100% de las filas.
- 0,0-0,2: 7,49% (n=1935) · 0,4-0,6: 17,56% (n=2569) · 0,8-1,0: 9,01% (n=677)

Conclusión: forma de campana: se compra menos lo más barato y lo más caro del rango buscado, más lo intermedio. Feature con señal real — usar la posición relativa, no solo price.

Encoding de categóricas nominales

- category (12), storage_type (3), brand (15), country_of_origin (10), allergens (7) — todas nominales, sin orden natural.
- One-hot encoding: N columnas binarias, evita imponer orden/magnitud falsos.
- country_of_origin muy desbalanceada: 75% (7500/10000) es "United States", el resto en 9 países con 245-331 filas c/u.

Conclusión: riesgo del desbalance en one-hot: categorías con pocos ejemplos dan poca evidencia para un peso confiable — puede ajustarse a ruido en vez de señal real.

country_of_origin vs. bought

país	% bought	n
Vietnam	16,61%	271
Italy	14,55%	275
Peru	13,88%	245
Mexico	13,77%	305
United States	12,99%	7500
Canada	12,73%	267
New Zealand	12,32%	284
Spain	12,08%	331
Chile	11,11%	261
Thailand	10,73%	261

Promedio general: 13,01%

- Rango angosto (10,7%-16,6%), sin país que se dispare.
- 9 países no-US con 245-331 filas c/u: diferencias de 2-3 puntos son compatibles con ruido (margen esperado por azar ± 4 puntos con $n \approx 245$).

Conclusión: sin señal clara — candidata a simplificar o evaluar en ablación.

nutrition_score vs. bought

- Correlación con bought prácticamente nula: -0,019.
- Tasa de bought por bins de 20 puntos: entre 12,1% y 14,1% (promedio general 13,01%), sin tendencia — ni "más nutritivo vende más" ni lo contrario.

Conclusión: sin señal en el cruce univariado — igual que `country_of_origin`, candidata a incluir/excluir en el estudio de ablación (podría aportar en combinación con otras features).

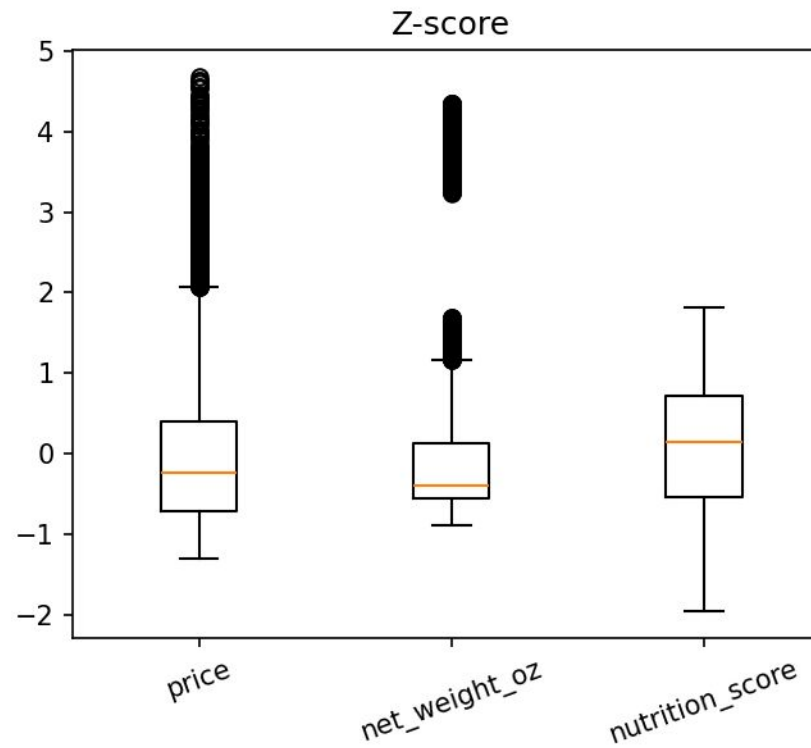
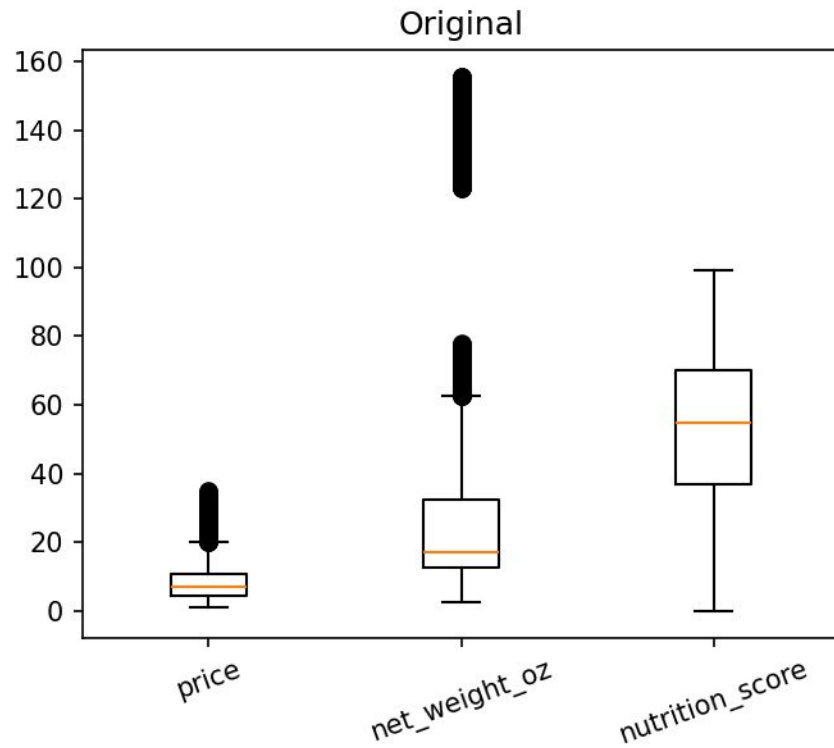
timestamp y query_id: sin señal encontrada

- timestamp: se probó si la hora del día predice bought (hipótesis: alcohol/Beverages a la noche, desayuno/Bakery a la mañana). Tasa por hora salta erráticamente entre horas consecutivas (ej. Beverages 21h=21,3% pero 20h=10,5%) — ruido, con solo 30-58 filas por hora dentro de cada categoría.
- query_id: el ID en sí no debe usarse (no generaliza). La feature derivada "n_resultados de la búsqueda" (2 a 8) tampoco mostró señal (tasa entre 12-15% sin importar el tamaño).

Conclusión: ambas descartadas como features. query_id sigue siendo necesario como clave para agrupar filas y calcular el BTR agregado por búsqueda.

Preprocesamiento numéricas: normalización z-score

Features numéricas antes/después de estandarizar (z-score)

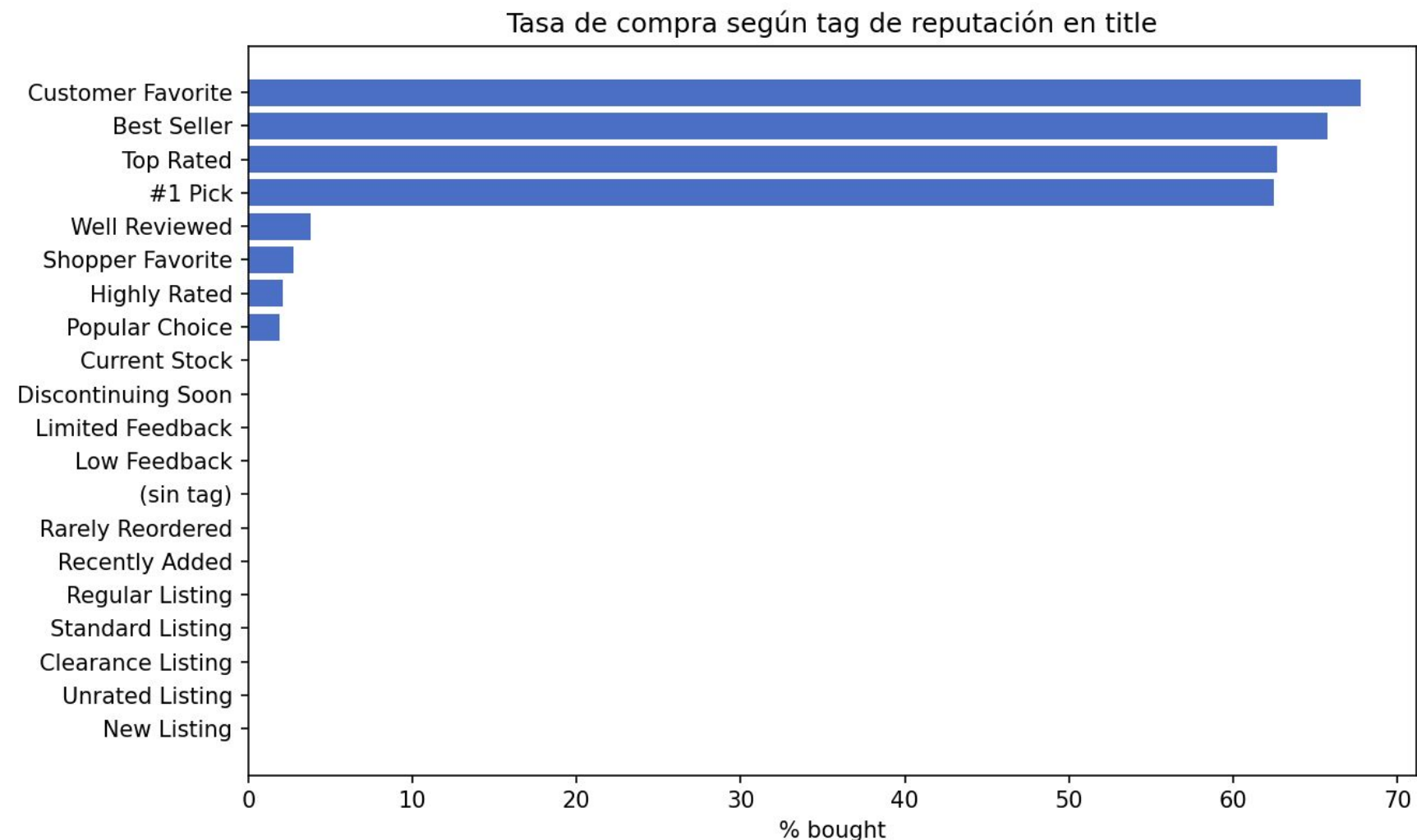


Parámetros de la corrida:

– price, net_weight_oz, nutrition_score
– z-score: $(x - \text{media}) / \text{desvío}$

- Escalas muy distintas: price (1,2-35), net_weight_oz (2,8-155), nutrition_score (0-99).

Hallazgo: tag de reputación oculto en title



Parámetros de la corrida:

- regex: `\(((^)+)\)\s*$` sobre title
- cruce con bought, 10.000 filas

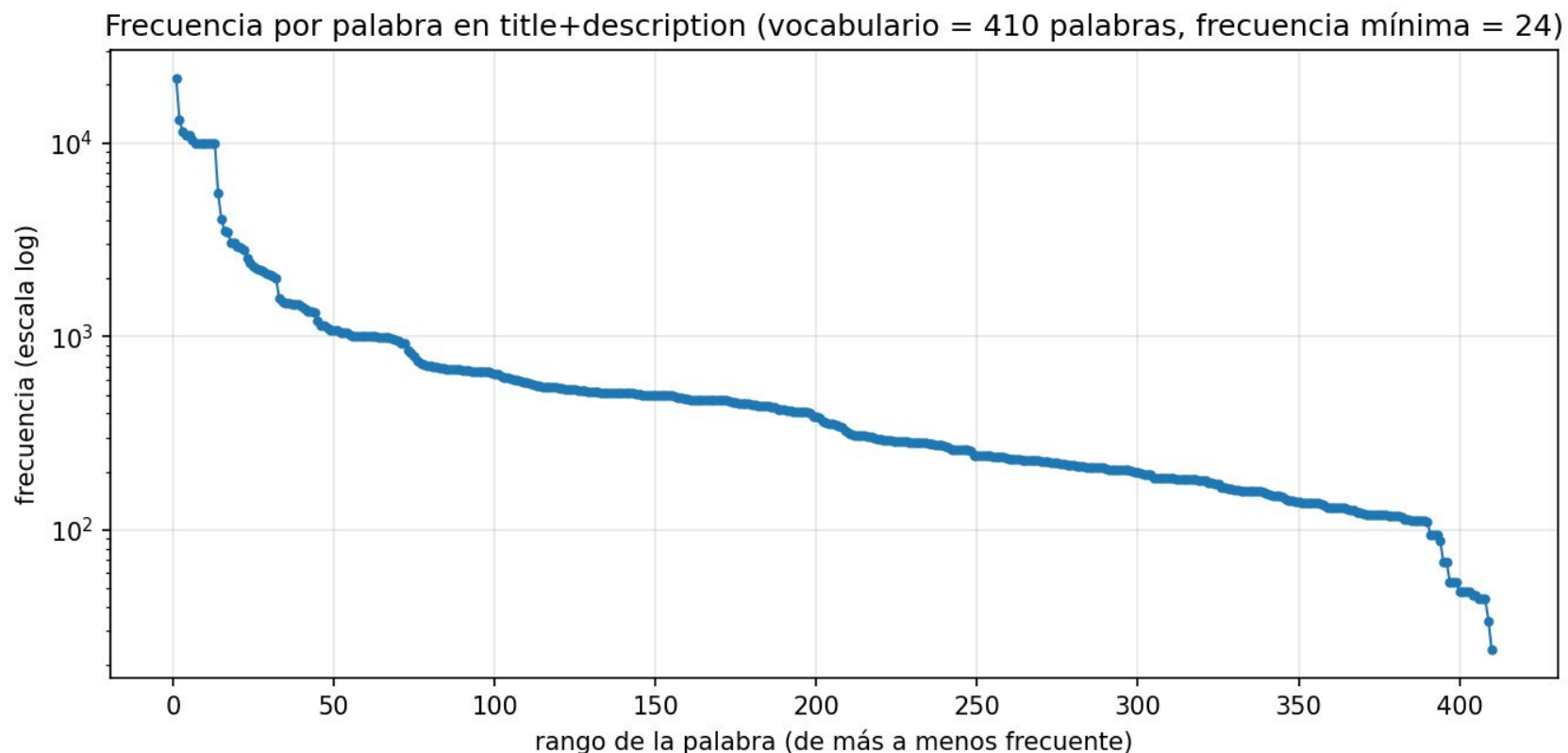
- Customer Favorite: 67,75% (n=493)
- Best Seller: 65,74% (n=470)
- Top Rated: 62,71% (n=472)
- #1 Pick: 62,50% (n=496)
- Well Reviewed / Shopper Favorite /
- Highly Rated / Popular Choice: 1,9-3,8%
- Las otras 11 (incl. sin tag): 0,00%

Conclusión: señal más fuerte de todo el EDA (base 13,01%). Se decide NO parsearla — dejar title crudo para que el Transformer la aprenda vía atención (foco: comprensión de la arquitectura, no un atajo).

Tokenización de title/description

Parámetros de la corrida:

- title + description concatenados, 10.000 filas
- minúsculas, split por no-alfanumérico, números como token propio



- El vocabulario de title+description es chico y cerrado: solo 410 palabras distintas, y todas se repiten muchas veces (ninguna aparece menos de 24 veces)
- El texto no es lenguaje libre — es texto generado por plantilla (frases fijas con nombres de producto/categoría/marca que se van completando)
- Con un vocabulario tan chico y repetitivo, no hace falta partir las palabras en pedazos más chicos

Lista final de features

- Descartadas (leakage): cart.
- Descartadas (redundantes): filter_category, filter_storage_type, package_size, unit_of_measure.
- Numéricas (z-score): net_weight_oz, posición relativa de price, dimensiones parseadas.
- One-hot: category, storage_type, brand, allergens.
- Multi-valor: ingredients (multi-hot o 12 combinaciones).
- Dudosas (a definir en ablación): country_of_origin, nutrition_score.
- Descartadas sin señal: timestamp, query_id (como feature).
- Para el Transformer: title, description — tokenización por palabra.

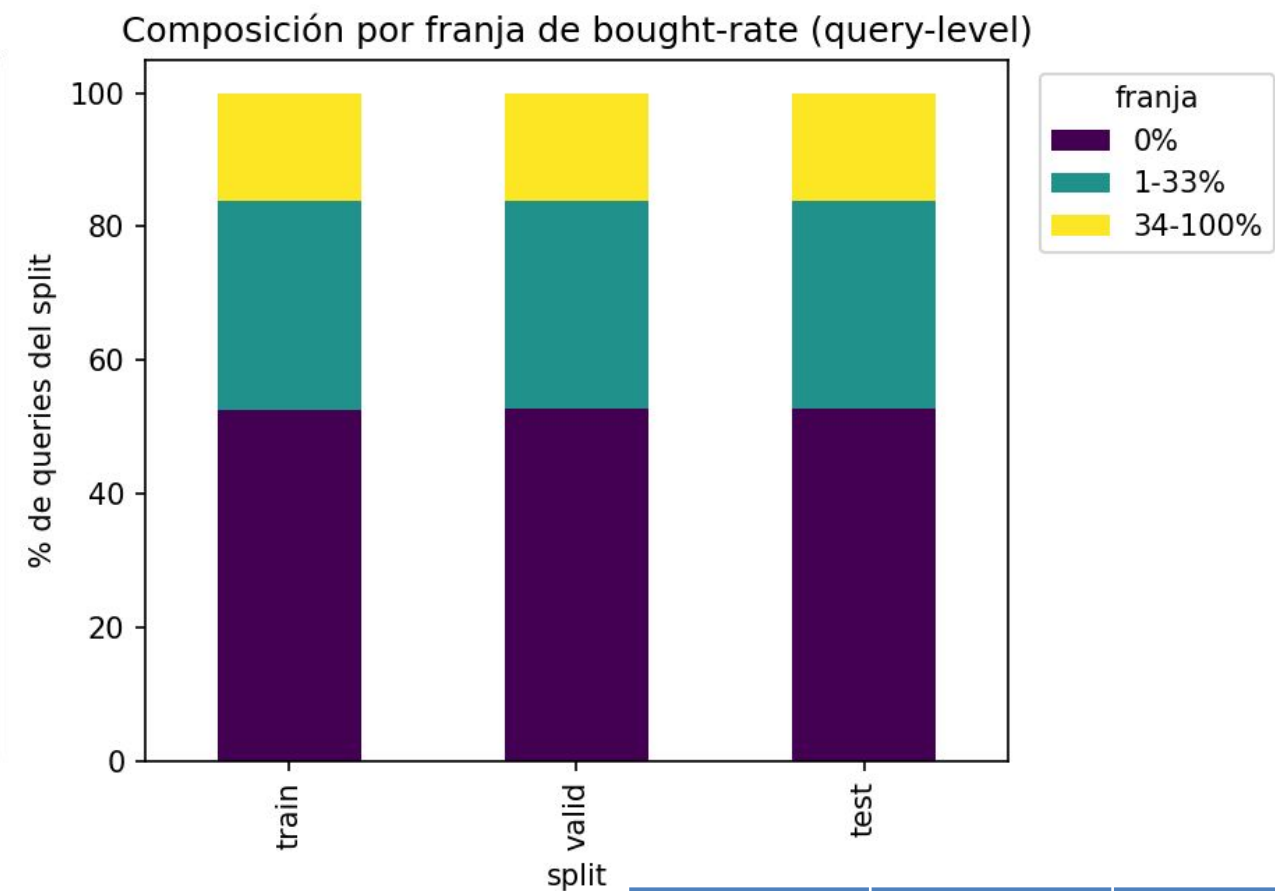
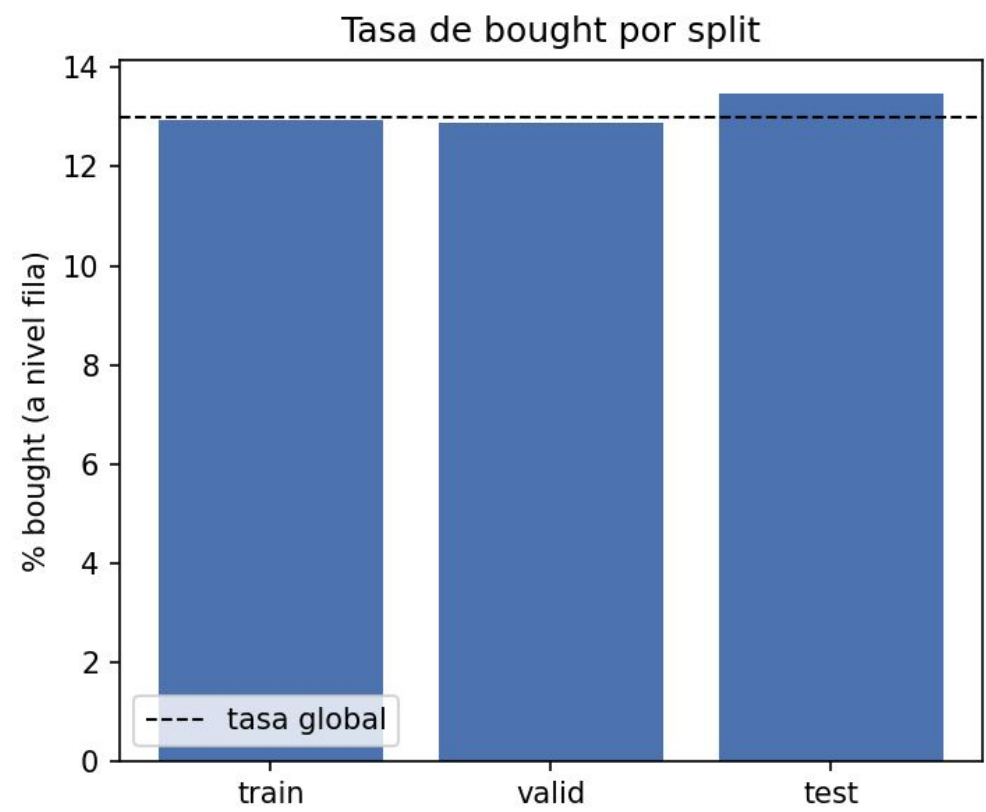
Ejercicio 2

Desarrollo del sistema

Split train / valid / test

- Cada búsqueda trae varios productos que comparten contexto (filter_category, filter_storage_type, filter_price_min, filter_price_max)
- Si se particiona fila por fila, esas filas no son observaciones independientes entre sí
- Repartirlas entre train y test podría inflar la métrica por esa correlación puntual entre filas, no por generalización real
- Partición por query_id completo: todas las filas de una búsqueda van al mismo split.
- **Supuesto:** query_id se usa como clave de agrupación de condiciones de búsqueda compartidas, sin asumir que representa un evento puntual.

Split: 70/15/15 estratificado por tasa de bought



Trade-off:

- Train más grande: el dataset es chico en términos absolutos (2012 queries, 10.000 filas) y bought está desbalanceado (13%) — más train significa más ejemplos positivos reales para aprender el patrón de compra.
- Un valid/test más grande dan una representación más real de la tasa de bought

proporción	distancia promedio a la tasa global — valid	distancia promedio — test
70/15/15	0,32 puntos	0,28 puntos
80/10/10	0,40 puntos	0,35 puntos

Cómo evaluamos el modelo

PR-AUC (área bajo la curva Precisión-Recall)

- **Precisión:** $TP / (TP + FP)$ (verdaderos positivos sobre todo lo que el modelo marcó como positivo).
- **Recall:** $TP / (TP + FN)$ (verdaderos positivos sobre todos los que realmente eran positivos).
- PR-AUC resume qué tan bien el modelo separa lo comprado de lo no comprado
- El punto de comparación relevante es la prevalencia de la clase positiva

ROC-AUC (área bajo la curva ROC)

- **Tasa de verdaderos positivos:** $TP / (TP + FN)$
- **Tasa de falsos positivos:** $FP / (FP + TN)$
- Probabilidad de que, tomando un producto comprado y uno no comprado al azar, el modelo le dé mayor probabilidad al comprado

Overfitting: Gap de PR-AUC (train – valid)

Arquitectura general del sistema

- Encoder-Decoder completo: para casos de traducción — no aplica.
- Decoder-only: generación de texto — no aplica
- Encoder-only para el texto (title + description) – **elegido**

Decision: Texto y datos tabulares se concatenan antes de la capa de salida.

- Permite desconectar el Transformer para análisis por separado
- El texto tiene una estructura de orden que justifica usar atencion.

Setup de entrenamiento

Semillas	0, 1, 2
Épocas	20

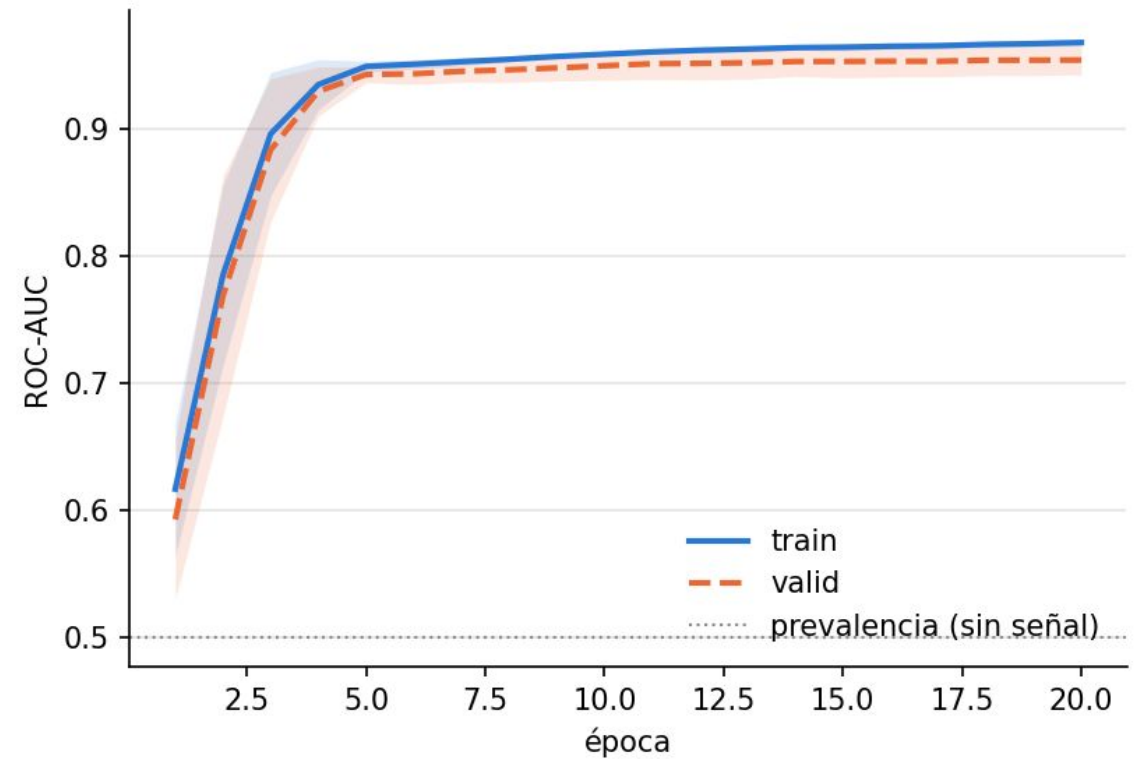
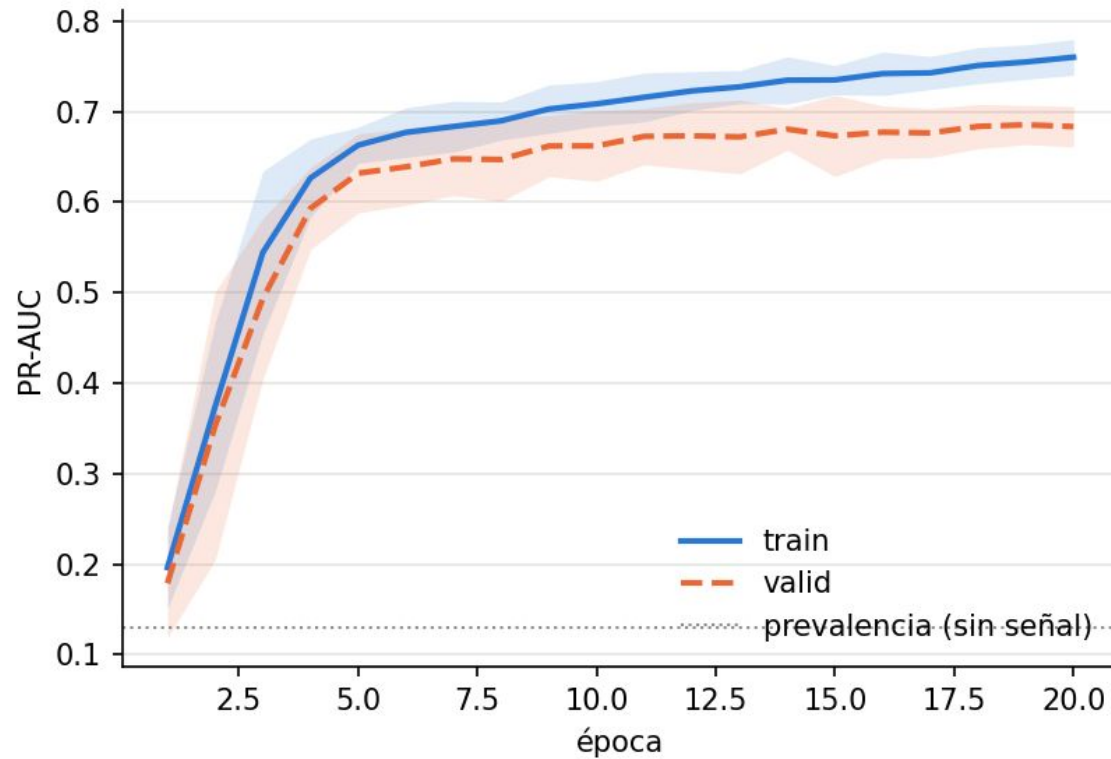
Experimento n°1

- Objetivo: Transformer mas simple posible

Arquitectura

- Embedding: $d_{\text{model}} = 16 \rightarrow$ empezamos chico
- $n_{\text{heads}} = 1$
- $n_{\text{layers}} = 1 \rightarrow$ un solo Encoder
- $\text{dim}_{\text{feedforward}} = 64 \rightarrow$ dimension 4x
- Positional encoding: senoidal $\rightarrow$ no agrega parámetros nuevos

Experimento 1 -- Transformer de texto puro (media $\pm$ desvío sobre 3 semillas)



valid PR-AUC	valid ROC-AUC	gap PR-AUC
0,688 $\pm$ 0,024	0,954 $\pm$ 0,012	0,058 $\pm$ 0,049

Conclusiones

- Hay señal fuerte en el texto solo
- Confirma lo visto en el ej 1: el tag de reputación embebido en title es predictivo
- Overfitting leve pero desparejo entre semillas: el gap va de 0,024 a 0,114
- El ROC-AUC es menos sensible al desbalance de clases

Experimento nº2 - aumento de heads

- Objetivo: Probar si separar distintos tipos de relación entre palabras ayuda a capturar mejor la señal, o si con un texto tan corto un solo head ya alcanza.

Arquitectura

- Embedding: d_model = 16

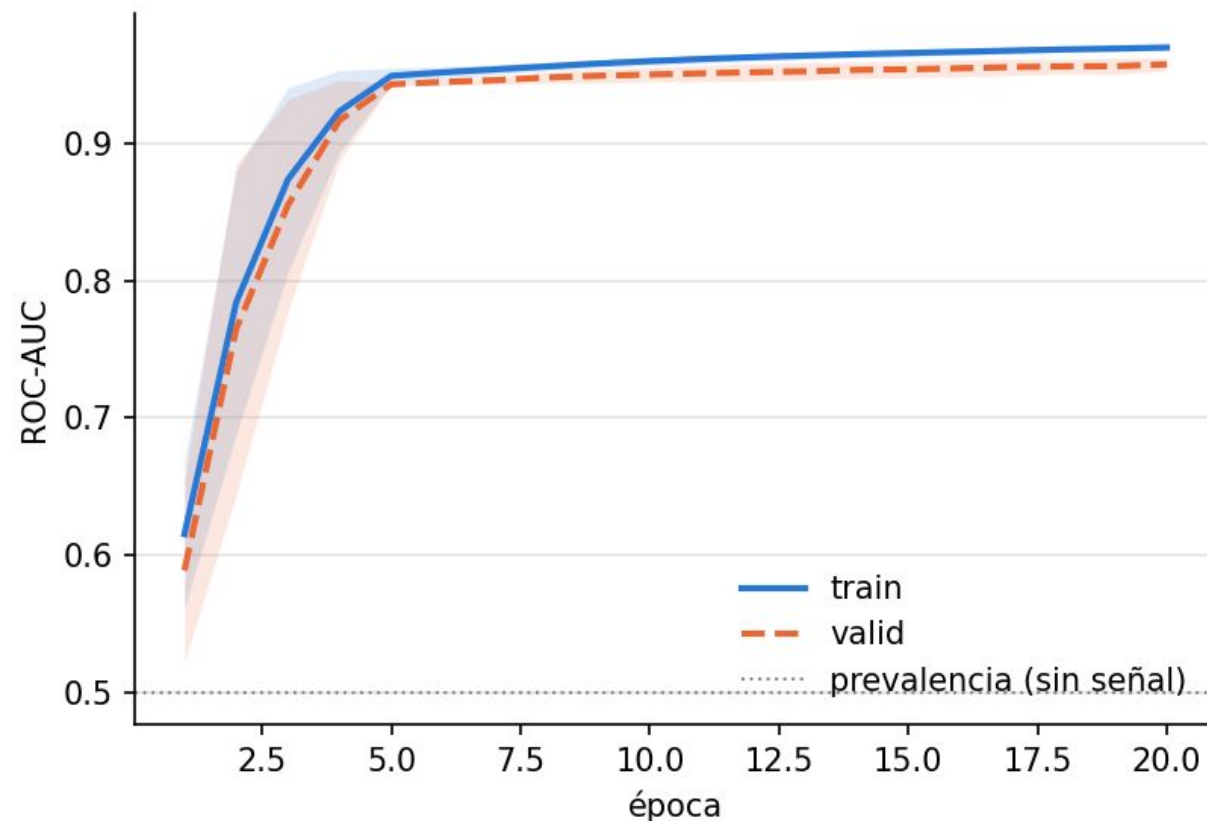
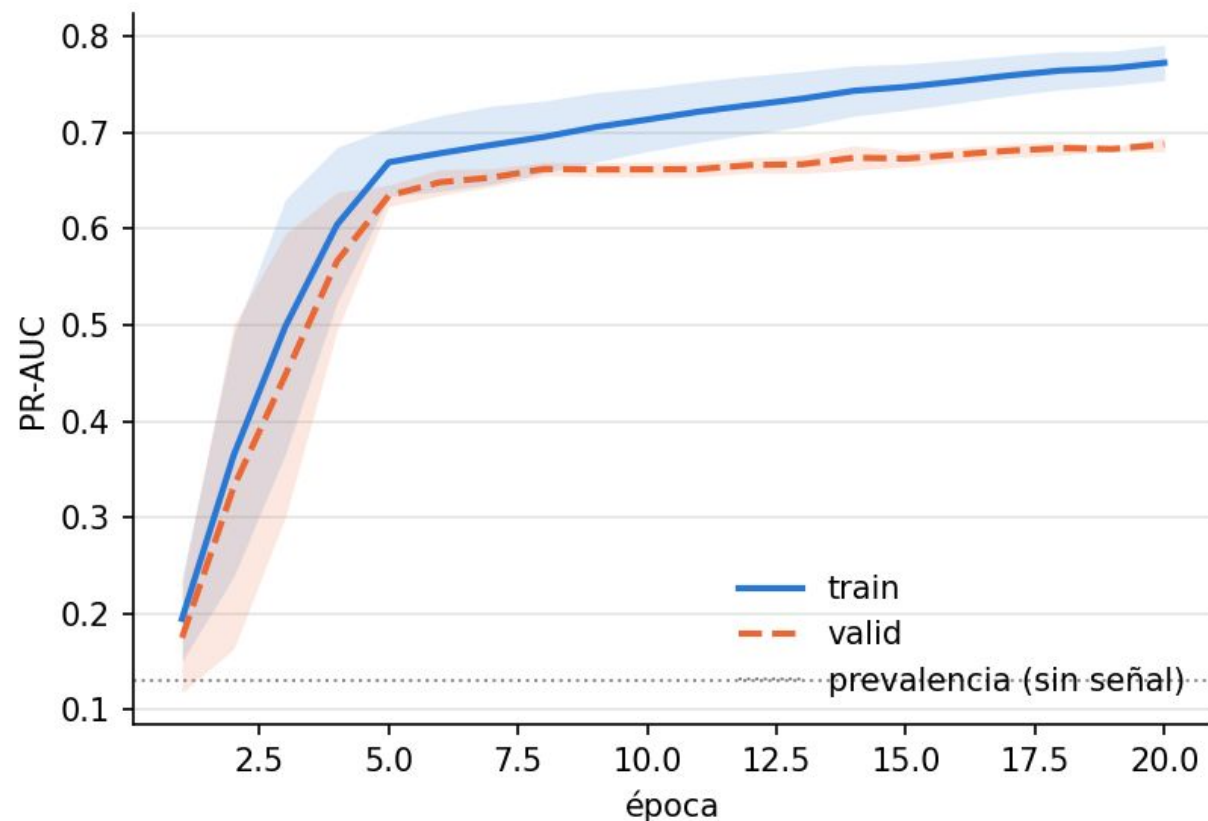
- n_heads = 2

- n_layers = 1

- dim_feedforward= 64

- Positional encoding: senoidal

Experimento 2 (media $\pm$ desvío sobre 3 semillas)



	Exp. 1 (n_heads=1)	Exp. 2 (n_heads=2)
valid PR-AUC	0,688 $\pm$ 0,024	0,688 $\pm$ 0,007
valid ROC-AUC	0,954 $\pm$ 0,012	0,957 $\pm$ 0,004
gap PR-AUC	0,058 $\pm$ 0,049	0,082 $\pm$ 0,015

- La performance pico no cambió (0,688 en ambos) $\rightarrow$ n_heads no agrega capacidad
- Cambió la estabilidad: el desvío estándar entre semillas bajó de 0,024 a 0,007
- En este rango de d_model, n_heads no es el cuello de botella de performance, es más un valor que afecta la estabilidad de entrenamiento que de capacidad

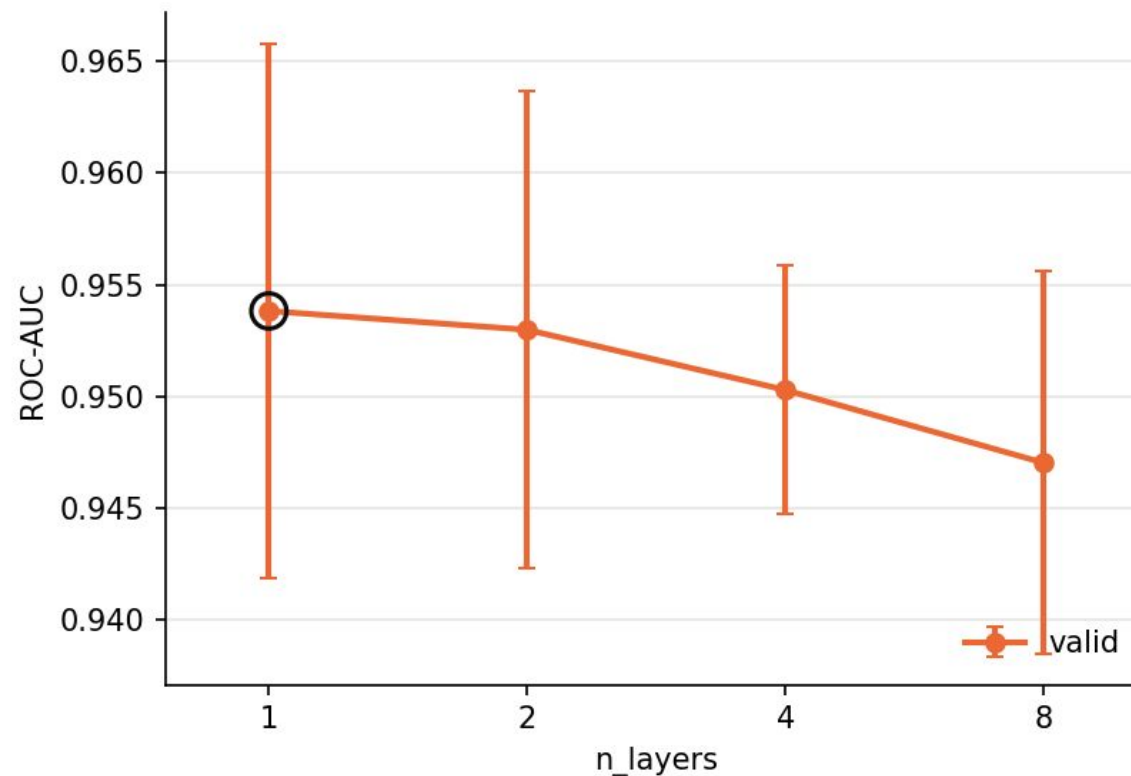
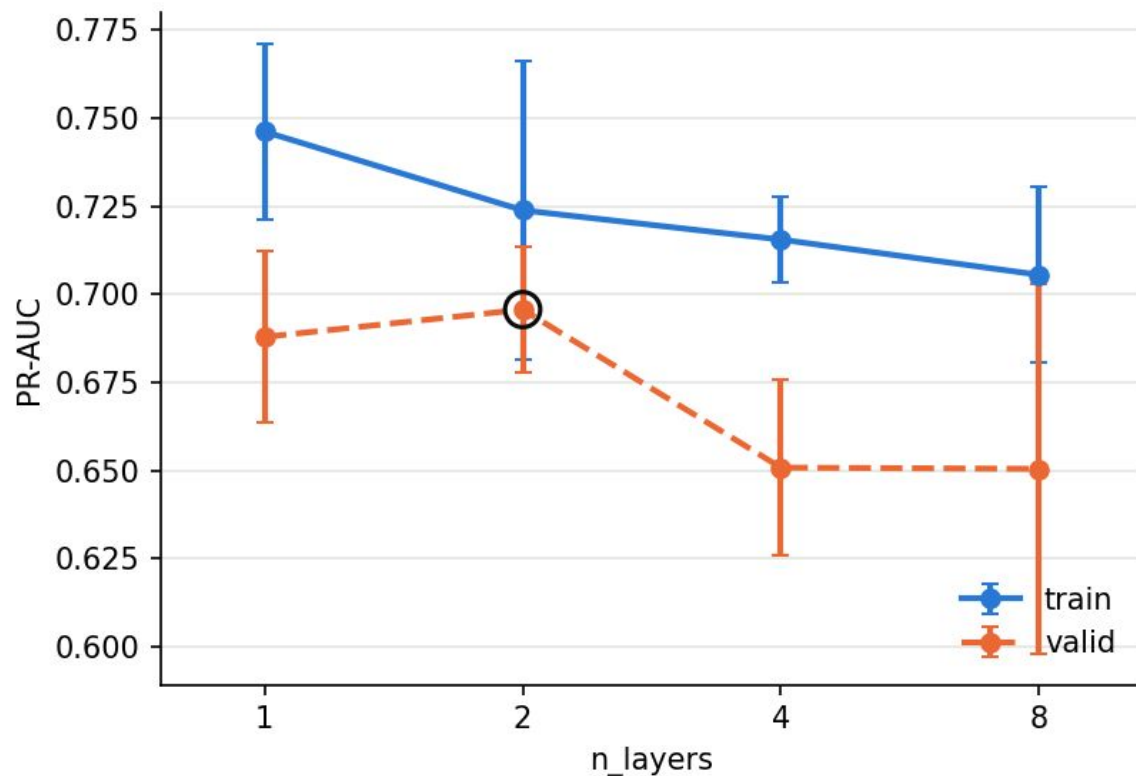
Experimento nº3 - layers

- Objetivo: Probar apilar diferentes numero de encoders

Arquitectura

- Embedding: $d_{\text{model}} = 16$
- $n_{\text{heads}} = 1$
- **$n_{\text{layers}} = 1 / 2 / 4 / 8$**
- $\text{dim}_{\text{feedforward}} = 64$
- Positional encoding: senoidal

Barrido de n_layers (círculo = mejor valor por PR-AUC de valid)



n_layers	valid PR-AUC	valid ROC-AUC	gap PR-AUC
1	0,688 ± 0,024	0,954 ± 0,012	0,058 ± 0,049
2	0,696 ± 0,018	0,953 ± 0,012	0,028 ± 0,042
4	0,651 ± 0,025	0,950 ± 0,006	0,065 ± 0,032
8	0,650 ± 0,053	0,947 ± 0,009	0,055 ± 0,035

- n_layers=2 gana en PR-AUC de valid y generaliza mejor
- En ROC-AUC, n_layers=1 queda apenas arriba
- Más profundidad no ayudó. Con un dataset chico y secuencias cortas, apilar más encoders agrega parámetros sin que haya más señal para aprovecharlos → modelo más difícil de optimizar de forma estable en pocas épocas.

Experimento n°4 - d_model

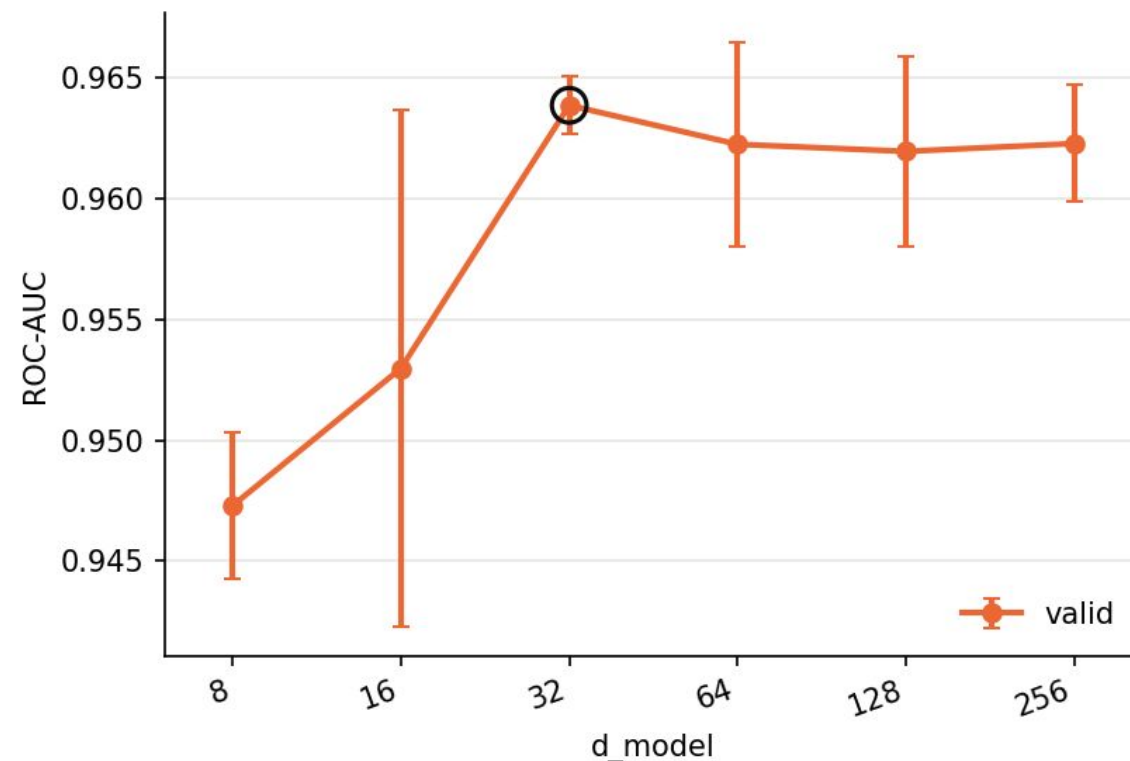
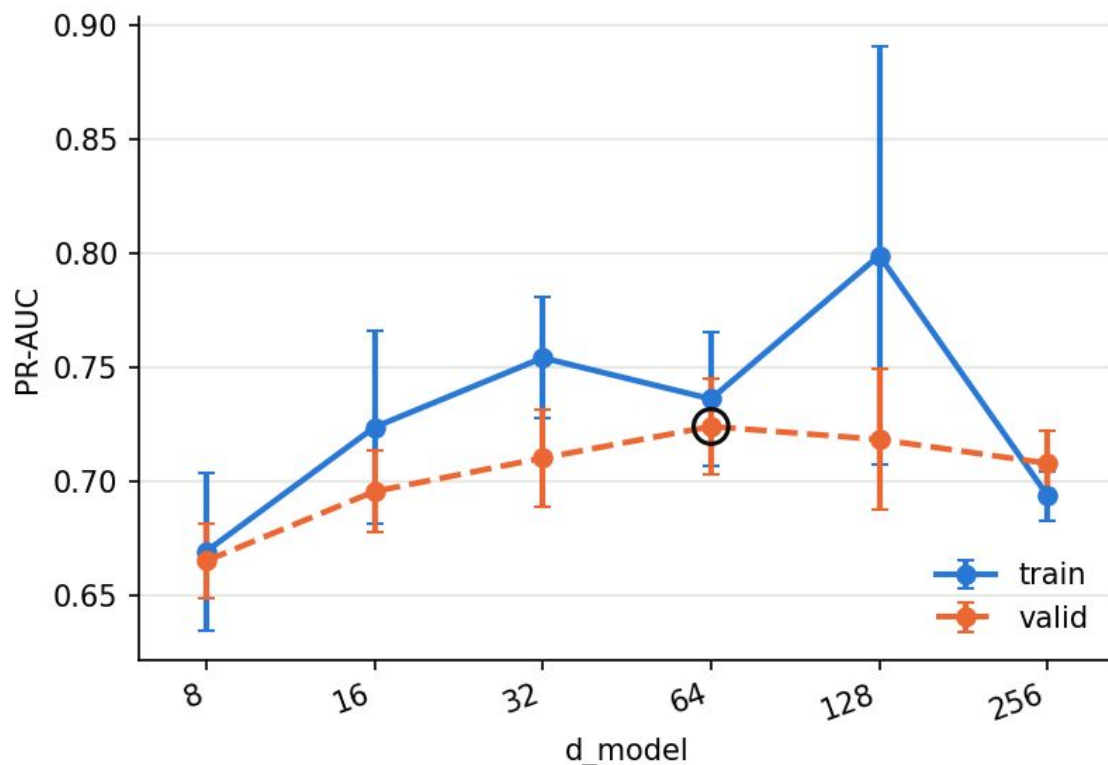
- Objetivo: ver si el modelo esta corto de capacidad de representación

Arquitectura

- **Embedding: d_model = 8 / 16 / 32 / 64 / 128 / 256**

- n_heads = 1
- n_layers = 2
- dim_feedforward= 64
- Positional encoding: senoidal

Barrido de d_model (círculo = mejor valor por PR-AUC de valid)



d_model	valid PR-AUC	valid ROC-AUC	gap PR-AUC
8	0,665 ± 0,016	0,947 ± 0,003	0,004 ± 0,039
16	0,696 ± 0,018	0,953 ± 0,011	0,028 ± 0,042
32	0,710 ± 0,022	0,964 ± 0,001	0,044 ± 0,047
64	0,724 ± 0,021	0,962 ± 0,004	0,012 ± 0,014
128	0,718 ± 0,031	0,962 ± 0,004	0,081 ± 0,063
256	0,708 ± 0,014	0,962 ± 0,002	-0,014 ± 0,008

- PR-AUC sube en todo el rango probado, sin meseta ni caída
- d_model=64 es el mejor y el overfitting es chico (1x con respecto a dim_feedforward).
- En d_model=128 el gap de overfitting se dispara sin señal real adicional que aprovechar.
- En d_model=256 el gap da directamente negativo, el modelo ni siquiera logra ajustar bien train en 20 épocas

Experimento n°5 - dim_feedforward

- Objetivo: probar cambiar el ancho del MLP interno de cada encoder

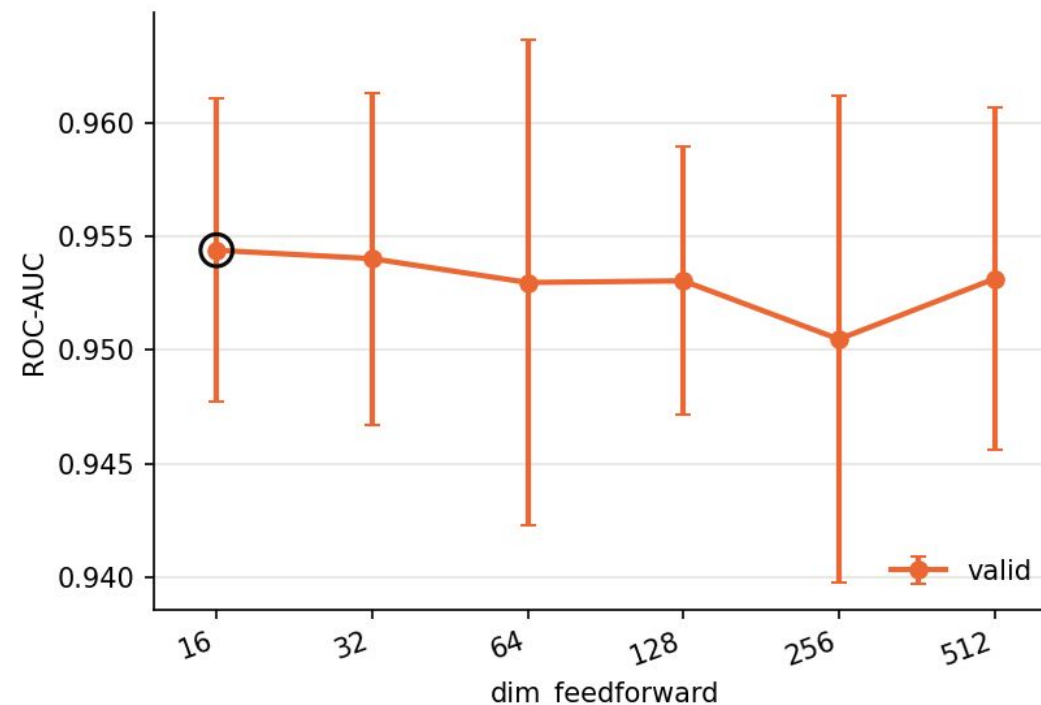
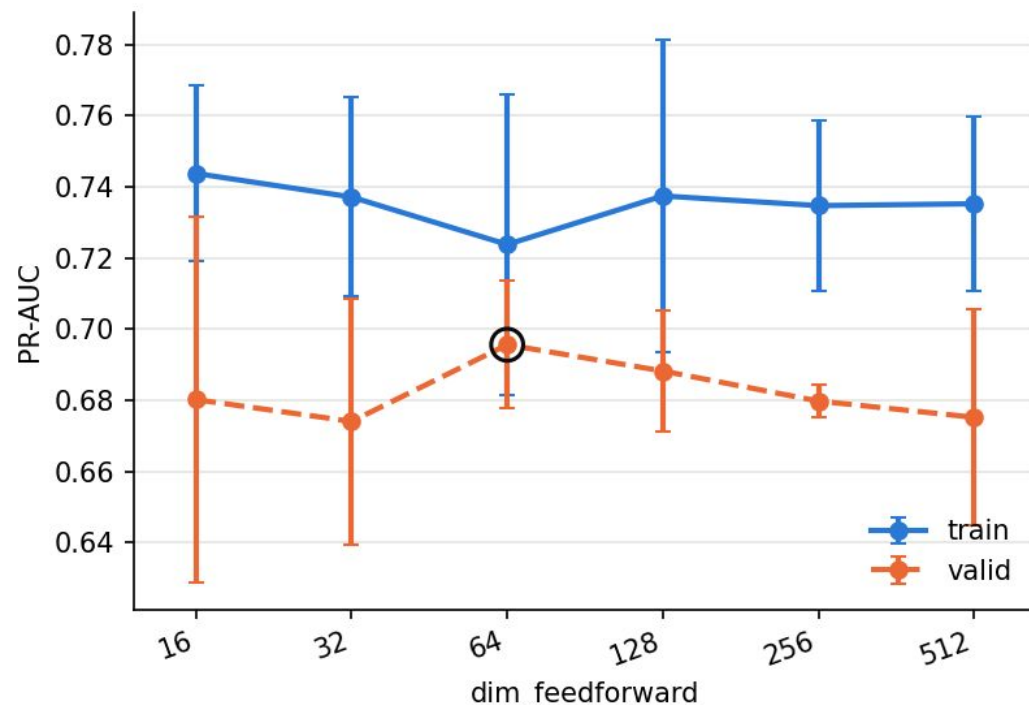
Arquitectura

- Embedding: d_model = 16
- n_heads = 1
- n_layers = 2

- dim_feedforward= 16 / 32 / 64 / 128 / 256 / 512

- Positional encoding: senoidal

Barrido de dim_feedforward (círculo = mejor valor por PR-AUC de valid)



dim_feedforward	valid PR-AUC	valid ROC-AUC	gap PR-AUC
16	0,680 ± 0,051	0,954 ± 0,007	0,064 ± 0,044
32	0,674 ± 0,035	0,954 ± 0,007	0,063 ± 0,027
64	0,696 ± 0,018	0,953 ± 0,011	0,028 ± 0,042
128	0,688 ± 0,017	0,953 ± 0,006	0,049 ± 0,058
256	0,680 ± 0,005	0,951 ± 0,011	0,055 ± 0,023
512	0,675 ± 0,030	0,953 ± 0,008	0,060 ± 0,045

- dim_feedforward=64 (proporción 4x sobre d_model=16) da mejor
- Todos los valores por encima (128, 256, 512) quedan peor
- Un feed-forward angosto es un cuello de botella y uno demasiado ancho agrega parámetros sin beneficio.

Experimento nº6 - d_model y dim_feedforward en conjunto

- Objetivo: analizar influencia de la proporción entre ambos parámetros

Arquitectura

- **Embedding: d_model = 32 / 64**

- n_heads = 1

- n_layers = 2

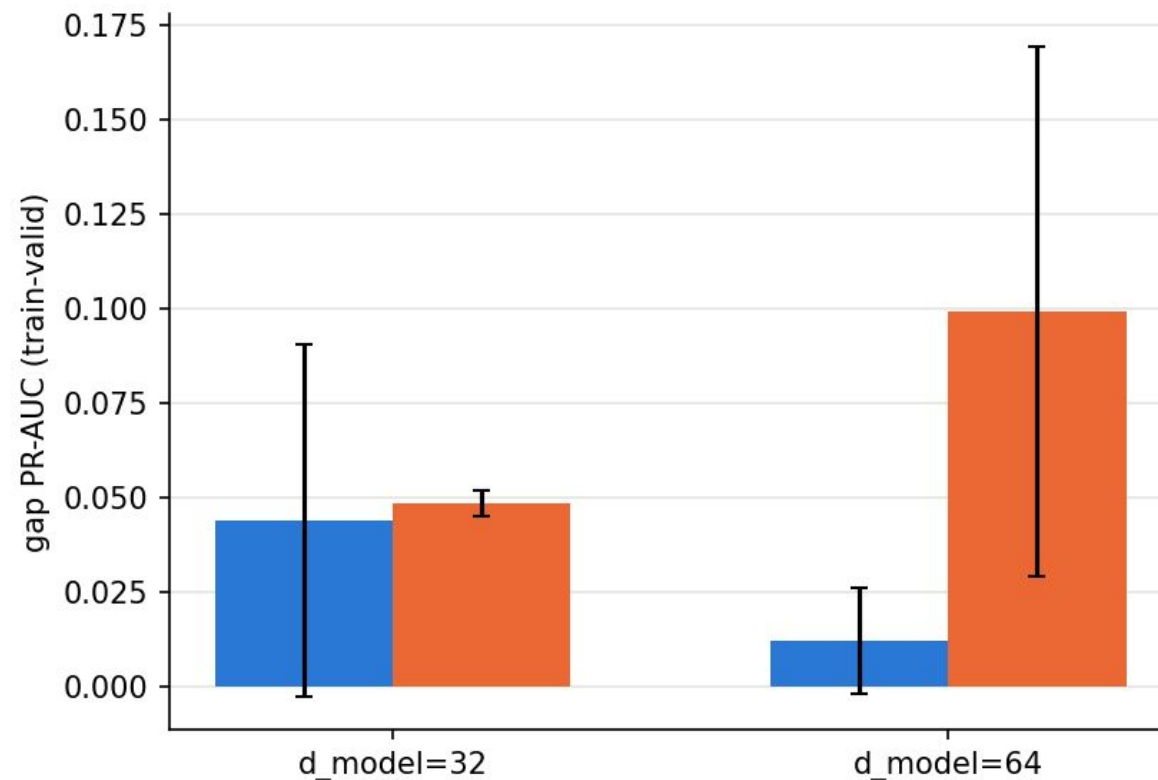
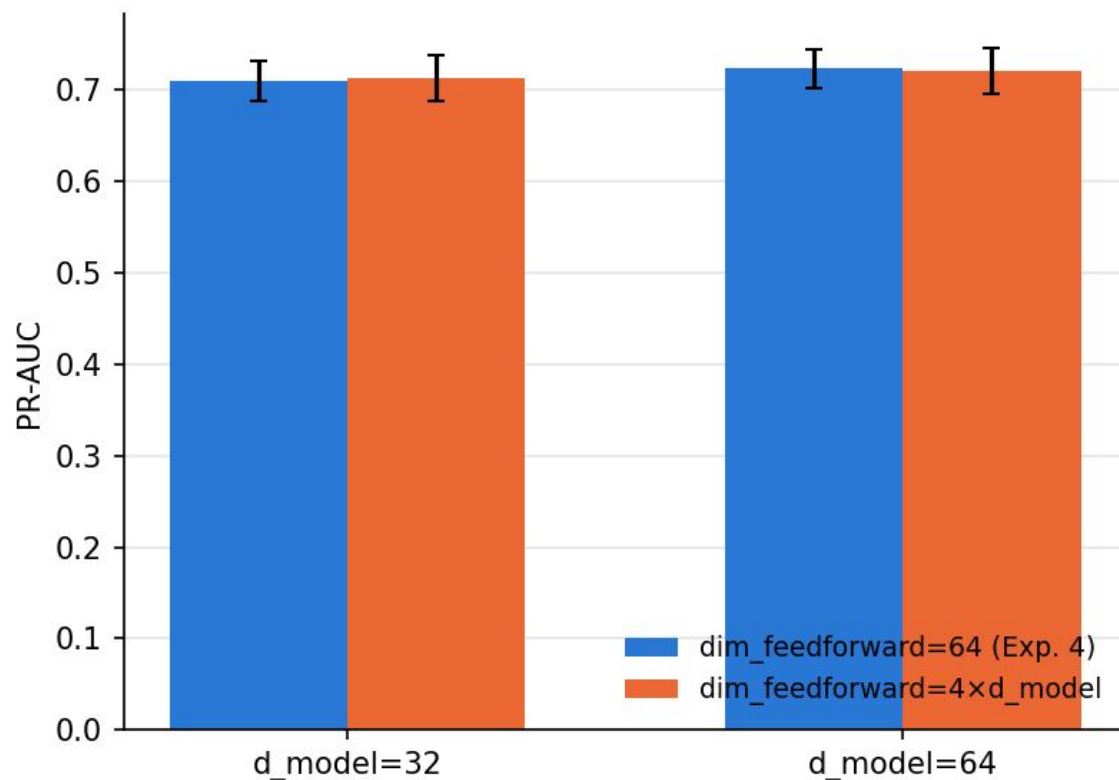
- **dim_feedforward= 64 /128 /256**

- Positional encoding: senoidal

d_model	dim_feedforward	proporción	valid PR-AUC	gap PR-AUC
32	64	2x	0,710 ± 0,022	0,044 ± 0,047
32	128	4x	0,713 ± 0,025	0,049 ± 0,004
64	64	1x	0,724 ± 0,021	0,012 ± 0,014
64	256	4x	0,721 ± 0,025	0,099 ± 0,070

- PR-AUC casi no cambia al escalar dim_feedforward
- Grafico izquierdo: las barras son prácticamente idénticas en altura.
- El feed-forward más ancho agrega capacidad que el modelo termina memorizando en vez de generalizar, sin que la performance de valid lo compense.
- Lo que pesa es la capacidad absoluta de d_model. Escalar también dim_feedforward no ayuda y en el punto más grande directamente perjudica la generalización.

Experimento 6 -- ¿escalar dim_feedforward junto con d_model ayuda?



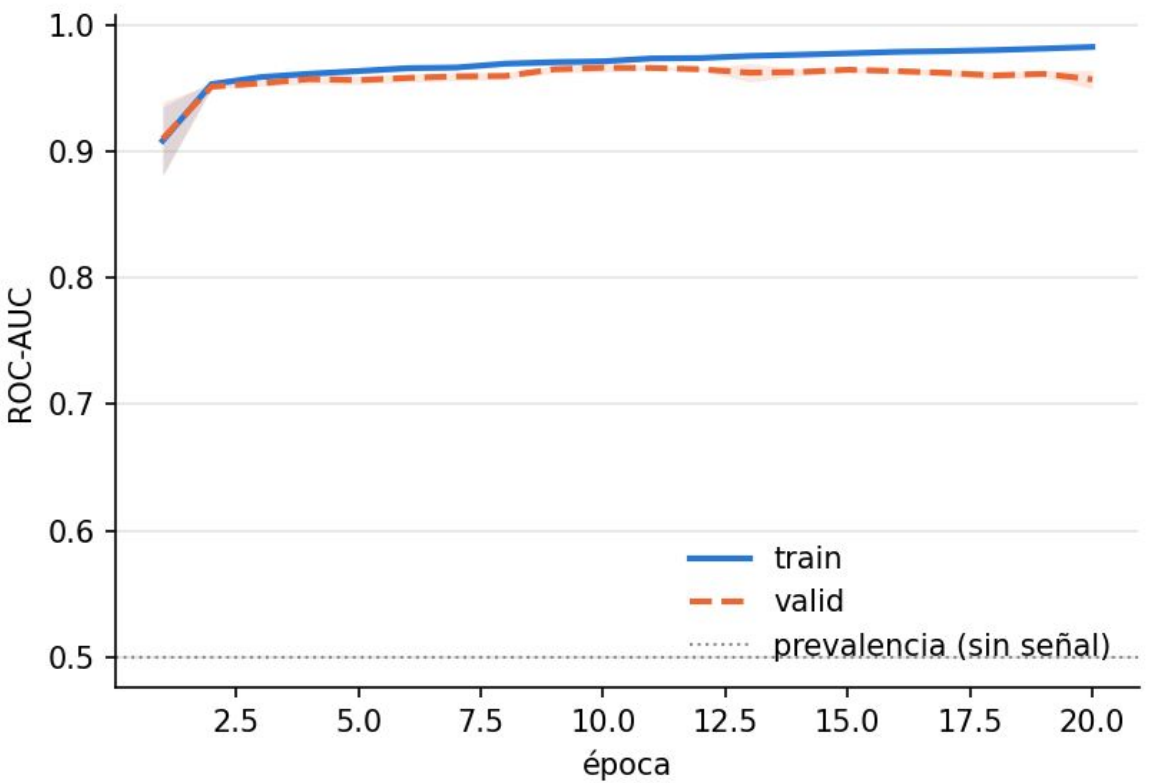
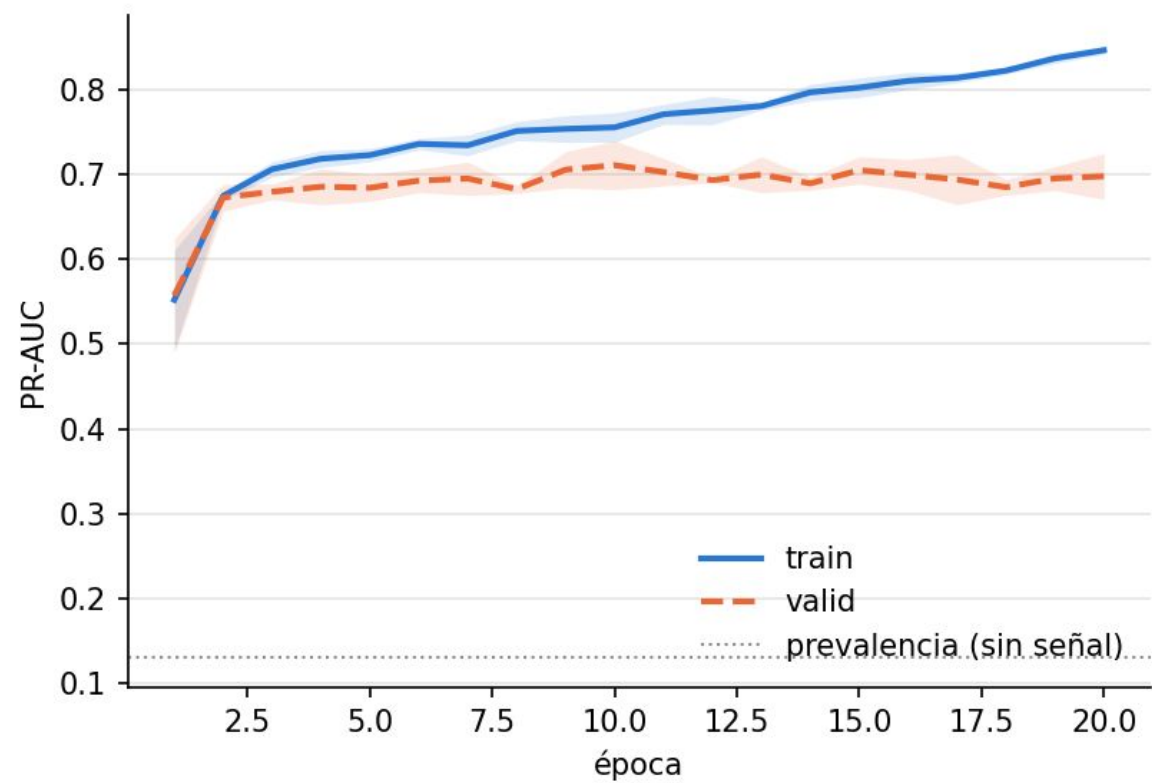
Experimento nº7 - sistema completo: texto + tabular

- Objetivo: analizar si concatenar el vector tabular al final mejora sobre el texto solo

Arquitectura

- Embedding: $d_{\text{model}} = 64$
- $n_{\text{heads}} = 1$
- $n_{\text{layers}} = 2$
- $\text{dim}_{\text{feedforward}} = 64$
- Positional encoding: senoidal
- Cabeza de salida: Linear

Experimento 7 (media $\pm$ desvío sobre 3 semillas)



	Solo texto	Texto + tabular
n_params	76.865	76.940
valid PR-AUC (media $\pm$ std)	0,724 $\pm$ 0,021	0,718 $\pm$ 0,023
valid ROC-AUC (media $\pm$ std)	0,962 $\pm$ 0,004	0,967 $\pm$ 0,002
gap PR-AUC (media $\pm$ std)	0,012 $\pm$ 0,014	0,045 $\pm$ 0,024
mejor época (típica)	~15-20	~10-13

- Agregar lo tabular no mejoró PR-AUC.
- Con solo 75 features tabulares más y una sola capa lineal para combinarlas con el texto, el modelo no encontró en ellas señal adicional
- ROC-AUC sí mejoró, las tabulares ayudan a ordenar mejor los casos en general
- El overfitting empeoró bastante, y aparece mucho antes. El gap casi se cuadruplica (0,012 $\rightarrow$ 0,045) y la mejor época baja de ~15-20 a ~10-13
- **Hipótesis:** la capa de salida, un solo Linear, no ve relación texto dato tabular

Experimento n°8 - capa oculta en la cabeza de salida

- Objetivo: Probar si el estancamiento del Experimento 7 se debía a la falta de una no-linealidad

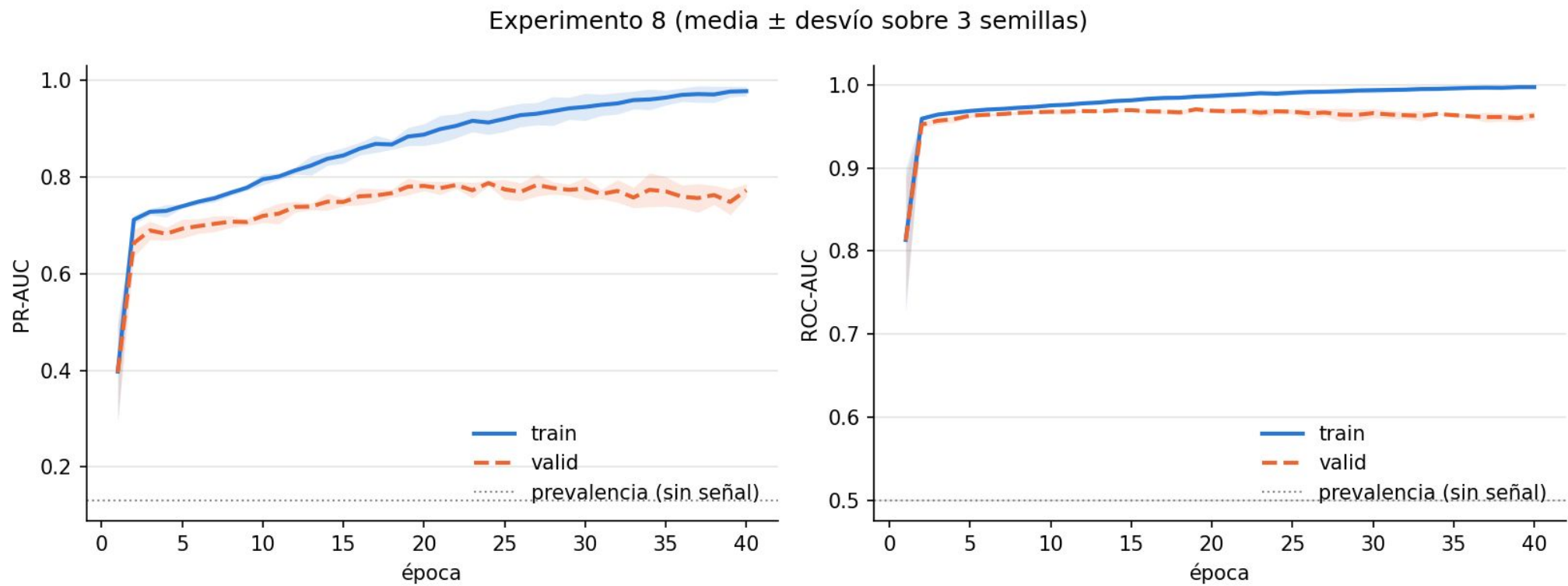
Arquitectura

- Embedding: $d_{\text{model}} = 64$
- $n_{\text{heads}} = 1$
- $n_{\text{layers}} = 2$
- $\text{dim}_{\text{feedforward}} = 64$
- Positional encoding: senoidal
- Cabeza de salida: $\text{Linear}(139 \rightarrow 64) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0,1) \rightarrow \text{Linear}(64 \rightarrow 1)$

Seguía subiendo sin señales de meseta, así que se repitió la misma configuración con **40 épocas**

	Solo texto (Exp. 4/6)	Texto + tabular, sin capa oculta	Texto + tabular, con capa oculta
n_params	76.865	76.940	85.825
valid PR-AUC (media ± std)	0,724 ± 0,021	0,718 ± 0,023	0,798 ± 0,008
valid ROC-AUC (media ± std)	0,962 ± 0,004	0,967 ± 0,002	0,968 ± 0,003
gap PR-AUC (media ± std)	0,012 ± 0,014	0,045 ± 0,024	0,128 ± 0,021

- La no-linealidad dio la mejor marca de todos los experimentos
- Con 40 épocas se ve el techo que con 20 no se veía: valid PR-AUC sube fuerte hasta las épocas 20-25
- La ganancia corriendo 20 vs 40 épocas es chica (0,791 vs 0,798) y viene de picos puntuales de alguna semilla
- 20 épocas da un resultado representativo



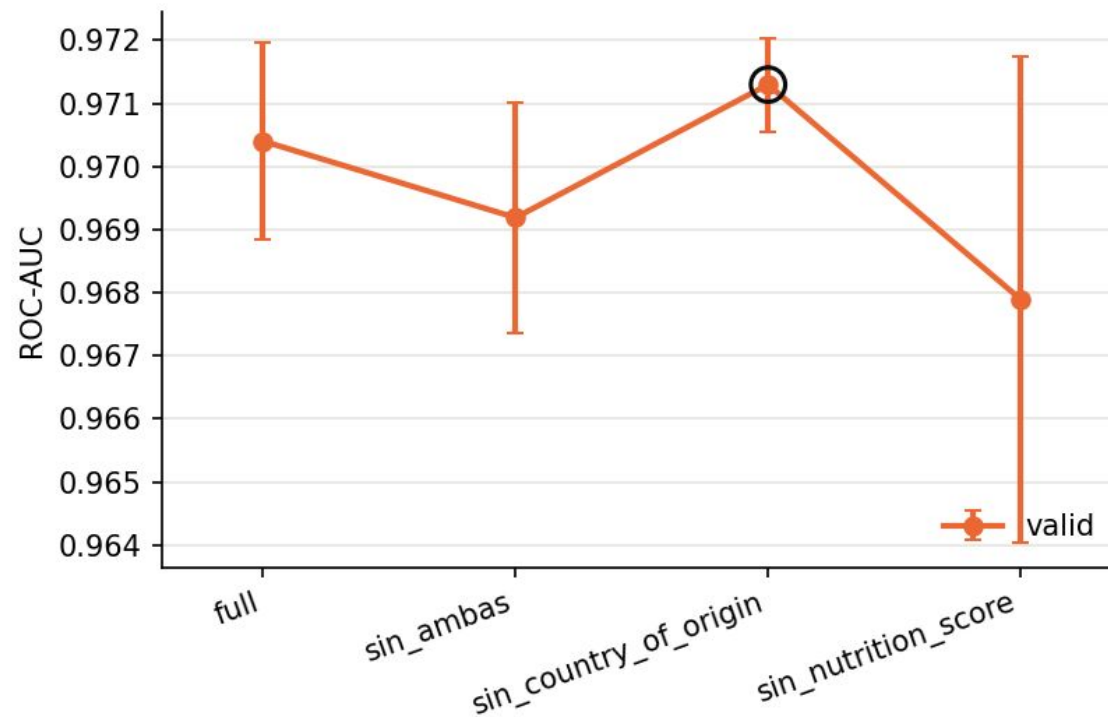
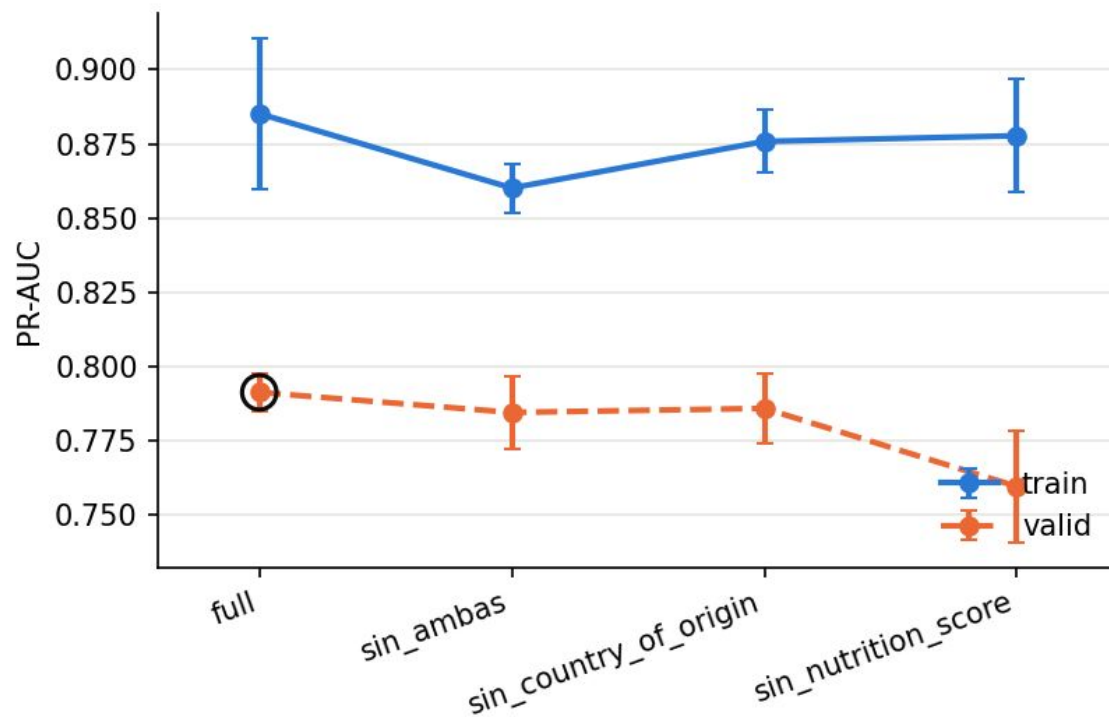
Experimento n°9 - country_of_origin y nutrition_score

- Objetivo: en el ejercicio 1 no encontramos señal clara para ninguna de las dos features. Testear su importancia

Arquitectura

- Embedding: d_model = 64
- n_heads = 1
- n_layers = 2
- dim_feedforward= 64
- Positional encoding: senoidal
- Cabeza de salida: Linear(139 $\rightarrow$ 64) $\rightarrow$ ReLU $\rightarrow$ Dropout(0,1) $\rightarrow$ Linear(64 $\rightarrow$ 1)

Barrido de variant (círculo = mejor valor por PR-AUC de valid)



variante	valid PR-AUC	gap PR-AUC
full	0,791 ± 0,006	0,094 ± 0,030
sin country_of_origin	0,786 ± 0,012	0,090 ± 0,023
sin ambas	0,785 ± 0,012	0,076 ± 0,014
sin nutrition_score	0,760 ± 0,019	0,118 ± 0,032

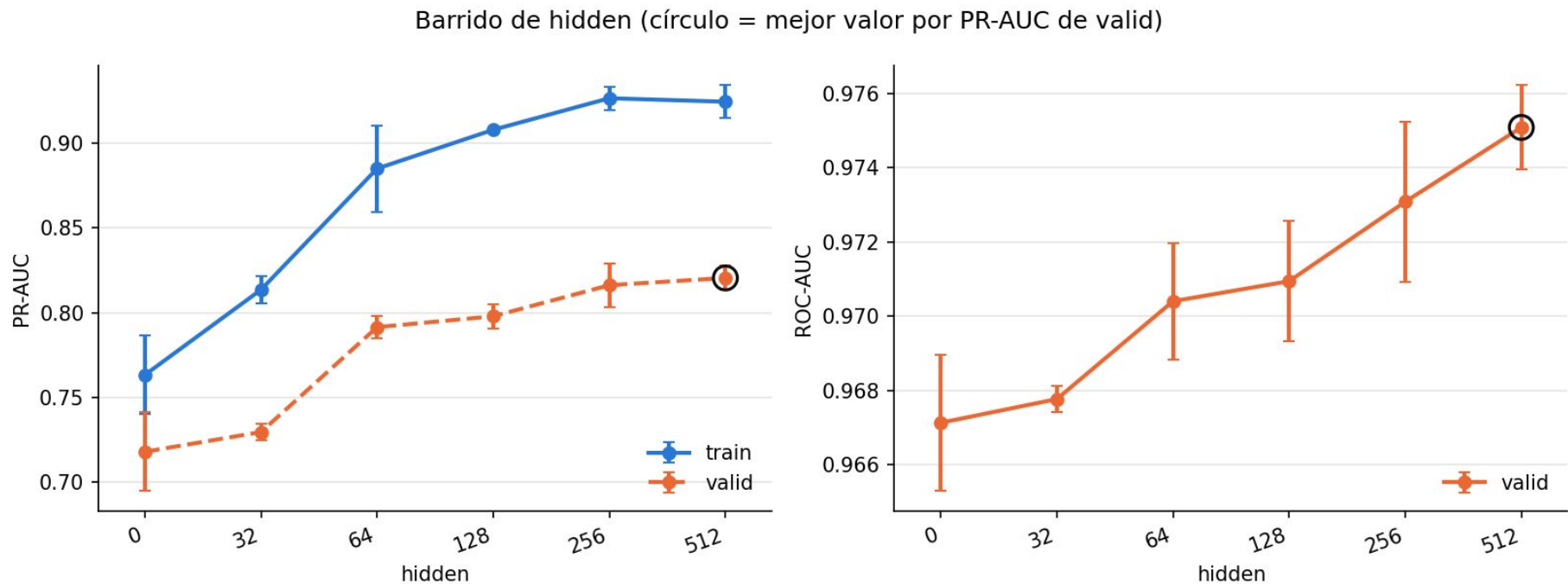
- Sacar country_of_origin apenas mueve el PR-AUC de valid ($0,791 \pm 0,006 \rightarrow 0,786 \pm 0,012$, una diferencia menor que los propios desvíos) y baja un poco el gap de overfitting
- Sacar nutrition_score sí aporta señal real ($0,791 \pm 0,006 \rightarrow 0,760 \pm 0,019$)
- **Decision:** se descarto country_of_origin achicando la cantidad de parametros

Experimento n°10 - ancho de la capa oculta

- Objetivo: Probar como variaciones del ancho de la capa oculta afectaba el rendimiento

Arquitectura

- Embedding: $d_{\text{model}} = 64$
- $n_{\text{heads}} = 1$
- $n_{\text{layers}} = 2$
- $\text{dim}_{\text{feedforward}} = 64$
- Positional encoding: senoidal
- Cabeza de salida: $\text{Linear}(139 \rightarrow \text{hidden}) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0,1) \rightarrow \text{Linear}(\text{hidden} \rightarrow 1)$



hidden	n_params	valid PR-AUC	valid ROC-AUC	gap PR-AUC
0	76.940	0,718 ± 0,023	0,967 ± 0,002	0,045 ± 0,024
32	81.313	0,729 ± 0,005	0,968 ± 0,000	0,084 ± 0,010
64	85.825	0,791 ± 0,006	0,970 ± 0,002	0,094 ± 0,030
128	94.849	0,798 ± 0,007	0,971 ± 0,002	0,110 ± 0,007
256	112.897	0,816 ± 0,013	0,973 ± 0,002	0,111 ± 0,020
512	148.993	0,820 ± 0,007	0,975 ± 0,001	0,104 ± 0,003

- Tanto train como valid se aplanan entre 256 y 512, después de subir sostenido en todo el resto del barrido.
- hidden=512 es la mejor marca de todo el estudio, pero por un margen chico sobre hidden=256.
- Nos quedamos con hidden=256

Experimento n°11 - positional encoding

- Objetivo: Ver importancia del orden del texto

Arquitectura

- Embedding: $d_{\text{model}} = 64$
- $n_{\text{heads}} = 1$
- $n_{\text{layers}} = 2$
- $\text{dim}_{\text{feedforward}} = 64$
- **Positional encoding: senoidal**

variante	valid PR-AUC	train PR-AUC	gap PR-AUC
con positional encoding	0,724 ± 0,021	0,736	0,012 ± 0,014
sin positional encoding	0,720 ± 0,001	0,893	0,174 ± 0,022

- PR-AUC de valid casi no cambia (0,724 → 0,720) — sacar el positional encoding no le hace perder al modelo casi nada de capacidad de alcanzar un buen resultado en valid.
- Tag de reputación en title (un patrón que parecía "posicional"), ese patrón puede ser más una cuestión de qué palabras aparecen juntas que de en qué posición exacta aparecen.
- Overfitting: el gap se dispara de 0,012 a 0,174
- **Decisión:** se mantiene positional encoding en la arquitectura final porque sin él el modelo sobreajusta muchísimo más rápido y más fuerte, lo cual sería un problema real en un dataset todavía más chico o entrenando más épocas.

Modelo final elegido

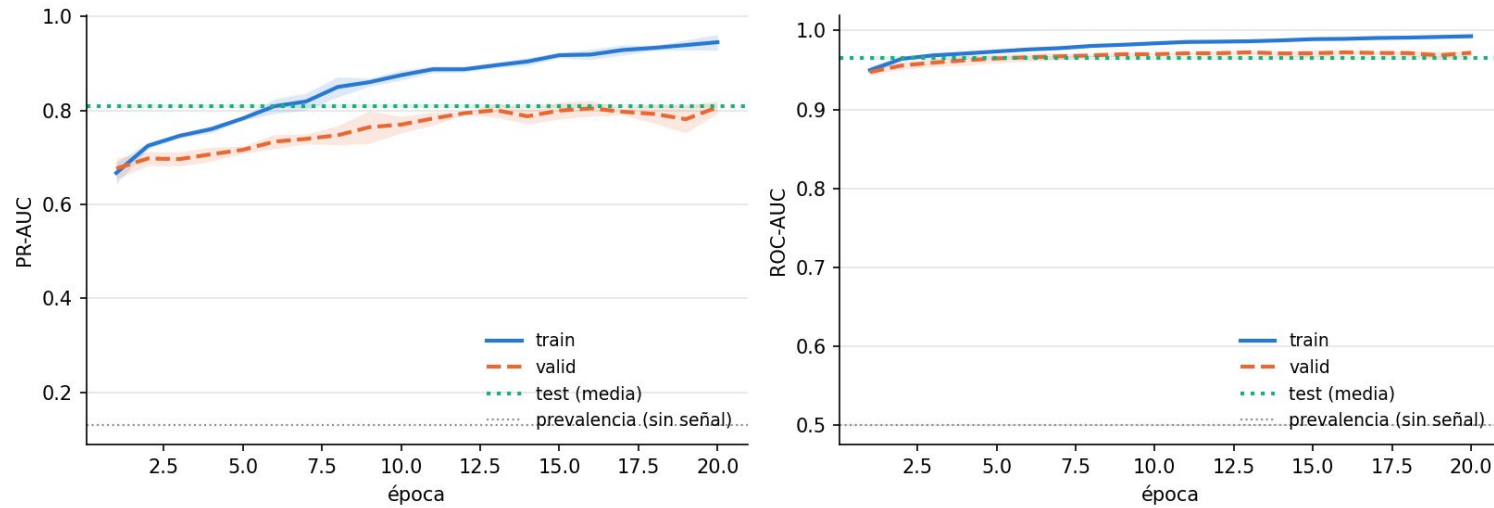
Arquitectura

- Embedding: $d_{\text{model}} = 64$
- $n_{\text{heads}} = 1$
- $n_{\text{layers}} = 2$
- $\text{dim}_{\text{feedforward}} = 64$
- Positional encoding: senoidal
- Cabeza de salida: $\text{Linear}(129 \rightarrow 256) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.1) \rightarrow \text{Linear}(256 \rightarrow 1)$

valid PR-AUC	valid ROC-AUC	gap PR-AUC
$0,816 \pm 0,013$	$0,973 \pm 0,002$	$0,111 \pm 0,020$

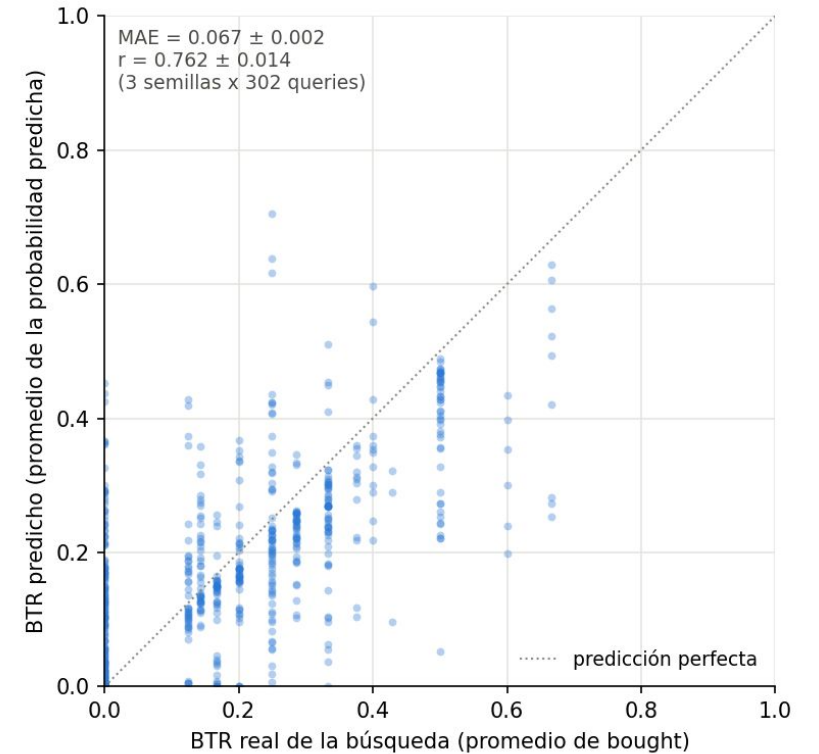
Evaluación en test

Configuración final -- train/valid por época + resultado de test (3 semillas)



seed	mejor época (por valid)	test PR-AUC	test ROC-AUC
0	16	0,806	0,962
1	13	0,813	0,966
2	15	0,809	0,967
media ± std	—	0,809 ± 0,003	0,965 ± 0,003

BTR por búsqueda en test: real vs. predicho



- El modelo agarra bien la tendencia general ($r=0,76$) y el error típico por búsqueda es chico (0,067).
- Tiende a sobreestimar el BTR, se ve la columna de puntos en $btr_real=0$ con predicciones dispersas (el modelo se entrena a nivel fila)

Ejercicio 3: Personalización

Problema actual:

- El dataset no tiene ningún identificador de usuario
- Cada fila se trata de forma independiente (no sabe si dos búsquedas distintas fueron hechas por la misma persona)
- Sin eso, no se puede adaptar el BTR

Propuesta:

- Un `user_id` que permita reconstruir el historial de compras pasadas de cada usuario

Cómo se integraría a la arquitectura ya construida:

- El historial de compras pasadas de un usuario es una secuencia — igual que `title/description` es una secuencia de palabras
- Se reutiliza la misma pieza ya construida para el texto: Encoder + pooling
- Cada producto pasado se convierte en un vector (embedding), igual que cada palabra
- La secuencia de esos vectores entra a un Encoder, que aplica self-attention entre las compras pasadas (en vez de entre palabras)
- El pooling resume esa secuencia en un solo vector: el "perfil" del usuario
- Ese vector se concatena junto con el vector de texto y el vector tabular, antes de la capa de salida
- No hace falta una arquitectura nueva: es la misma idea aplicada a una secuencia distinta

Gracias